



Figure 2: PicoC running in batch and interactive modes

```
tar -xjvf picoc-version.tar.bz2
cd picoc-version
make
```

Optionally, you can also run *make test* to run tests distributed with PicoC. If the compilation succeeds, you should see a *picoc* executing in the source directory—copy that to any of the standard locations like */usr/local/bin*, or add the absolute path to the source directory to the *PATH* environment variable, by adding the following line to your *.bashrc* file:

```
PATH=$absolute path to PicoC source directory:$PATH
```

To try the interactive mode, run *picoc -i* and at the *picoc* prompt, enter the code statements from the file *testpicoc.c* (in the accompanying Zip archive, as no *testpicoc.c* was shown below). To interpret *testpicoc.c*, type *picoc testpicoc.c*.

Figure 2 shows a screen-shot of PicoC running in interactive and batch modes.

EiC

EiC stands for extensible interactive C. It is an interactive C code interpreter that is able to communicate with external applications. EiC provides some very useful features like pointer safety, and some extra C programming constructs that are not found in any C code interpreters.

To build and install EiC, fetch the source tarball from the project page, and run the following:

```
tar -zxvf EICsrc-version.tar.gz
cd eic-version
./config/makeconfig
make install
```

Add the following EiC-specific settings in *.bashrc*:

```
EICBASE=$absolute path to the EiC source directory>
export HOME/EiC-$EICBASE
```

You can start EiC in interactive mode with the *eic* command.

In addition to the interactive and batch modes, EiC can also run in a scripting mode, similar to shell scripts, where you

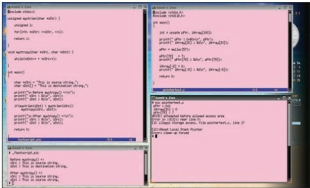


Figure 3: Scripting mode and pointer safety feature of EiC

can put various C code statements in a file, with or without a *main()* function. To try it out, run the *testscript.eic* file (from the accompanying Zip archive), first making it executable:

```
chmod uwx testscript.eic
./testscript.eic
```

Note one difference in the source file—instead of the usual C code, we have *#include stdio.h*. Due to EiC's extended features, you can also include headers without angle brackets.

Figure 3 shows *testscript.eic* and its output, in the two terminal windows on the left.

Now let's enjoy some unique features that are not provided by other C code interpreters. EiC can memorise all the C code statements entered at its interactive prompt—it creates a file named *EiChist.lst* in which it stores all commands that were interpreted without any error.

By default, *EiChist.lst* is created afresh on every interactive-mode run, in your current directory (*pwd*). You can alter this behaviour with the *-n* switch. If you launch EiC with the *-r* switch, then it runs (interprets) all the commands in *EiChist.lst* in sequence before presenting you with the prompt. This lets you continue from your previous EiC session, without having to re-define functions, etc. You can combine the two switches to create a static 'initialisation' set-up for EiC, where your preferred statements are run before your interactive session, and the history file is not overwritten.

The very useful *-R* switch works like *-r*, except that EiC asks you to choose whether to run each command from *EiChist.lst*. The options are Y (yes), N (no) or E (edit the line before running it).

Another great EiC feature is that it is pointer-safe. Any time you try to violate stack or heap memory limits through pointers, EiC will catch that and throw an error. This feature lets you verify your C programs before building and distributing, and will save you from debugging hard-to-find memory and pointer bugs. Interpret *pointertest.c* in the batch mode to see how EiC throws errors and stops as the first pointer limits violation. Figure 3 shows two terminal windows on the right—one with the *pointertest.c* source, and one with the results of running it.

The pointer safety feature is turned on by default, but